Lab 1

2.3.1 Analytical Calculation

Hand in your calculations of these two integrals. Show all work.

Formula 1

$$\int_0^{2\pi} \sin^2(5t)\, dt$$

✓ **Answer**

$$\int_0^{2\pi} 0.5 - 0.5\cos(10t)\, dt$$
$$= 0.5t - 0.05\sin(10t)\Big|_0^{2\pi}$$
$$= \pi$$

Formula 2

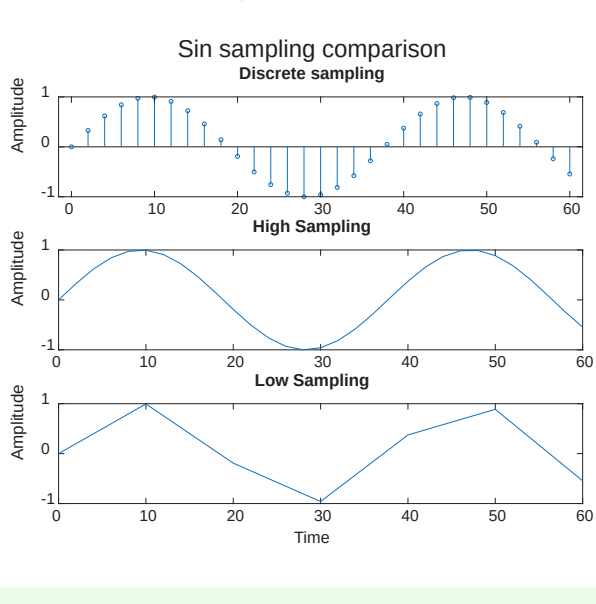
$$\int_0^{1} e^t\, dt$$

✓ **Answer**

$$= e^t\Big|_0^1$$
$$= e - 1$$

2.3.2 Displaying Continuous-Time and Discrete-Time Signals in Matlab

Submit a hard copy of the plots of the discrete-time function and two continuous-time functions. Label them with the title command, and include your names. Comment on the accuracy of each of the continuous time plots.



✓ **Answer**

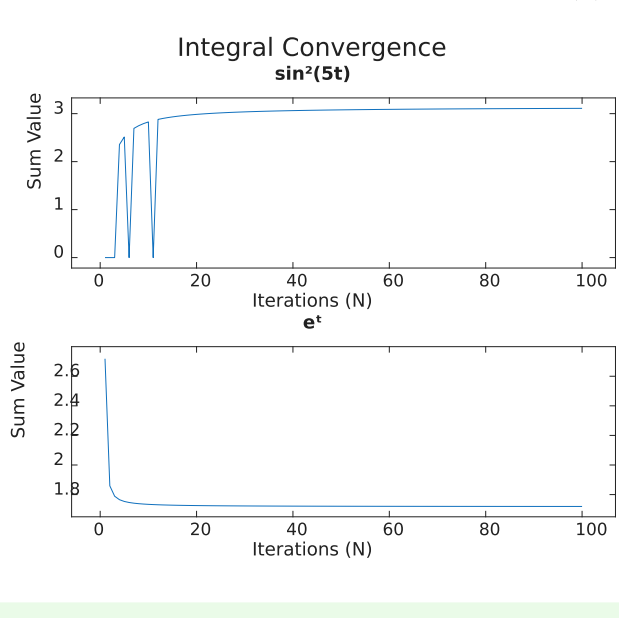
The higher sampling better tracked the waveform and accurately represented the underlying function.

The low frequency plot missed a lot of finer details of the function.

2.3.3 Numerical Computation of Continuous-Time Signals

Submit plots of $I(N)$ and $J(N)$ versus N. Use the subplot command to put both plots on a single sheet of paper. Also submit your Matlab code for each function.

Compare your results to the analytical solutions from the "Analytical Calculation" (Section 2.3.1: Analytical Calculation) section. Explain why $I(5) = I(10) = 0$.



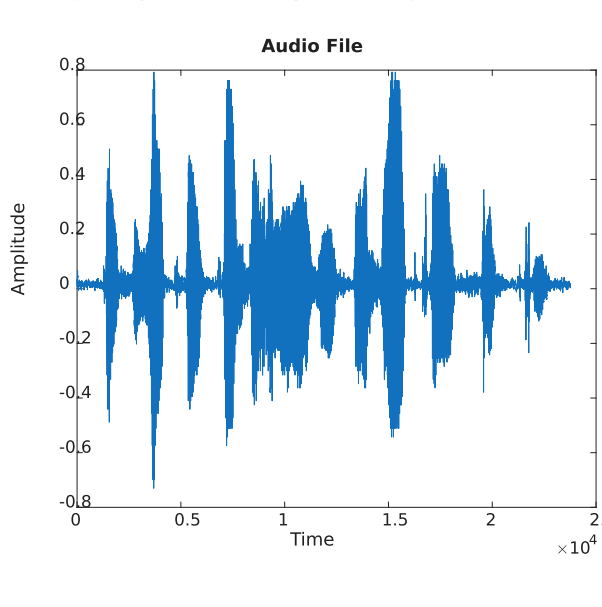
✓ **Answer**

Yes, they do converge to their analytical solutions with high accuracy as iterations increase.

$I(5) = I(10) = 0$ because at these sampling frequencies since every riemann rectangle is calculated at a zero point on the original signal, leading its sum to be zero overall.

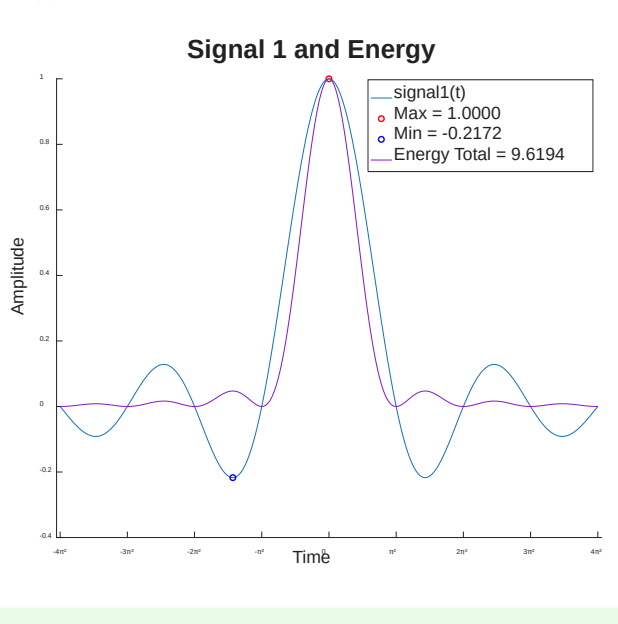
2.4 Processing of Speech Signals

Submit your plot of the speech signal.



2.5 Attributes of Continuous-Time Signals

Submit a plot of the function, and the computed values of the min, max, and energy. Explain your choice of the sampling period, and the starting and ending times. Also, submit the code for your energy function.

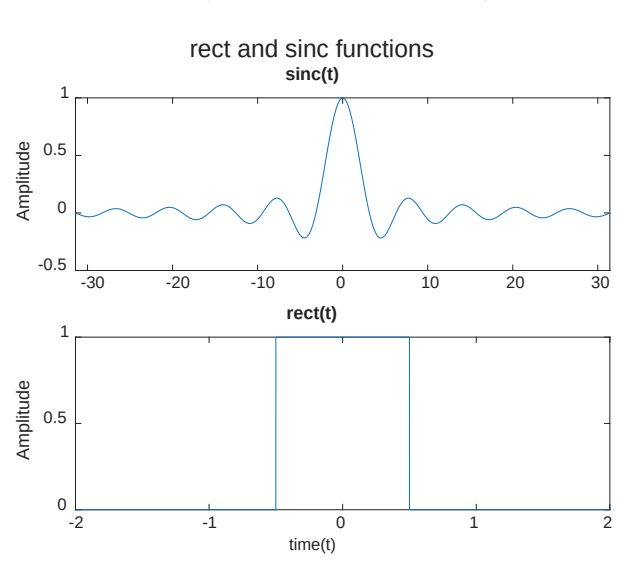


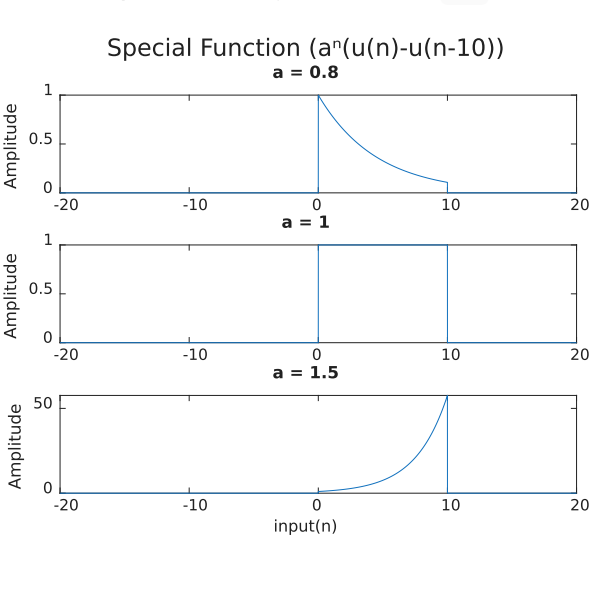
✓ **Answer**

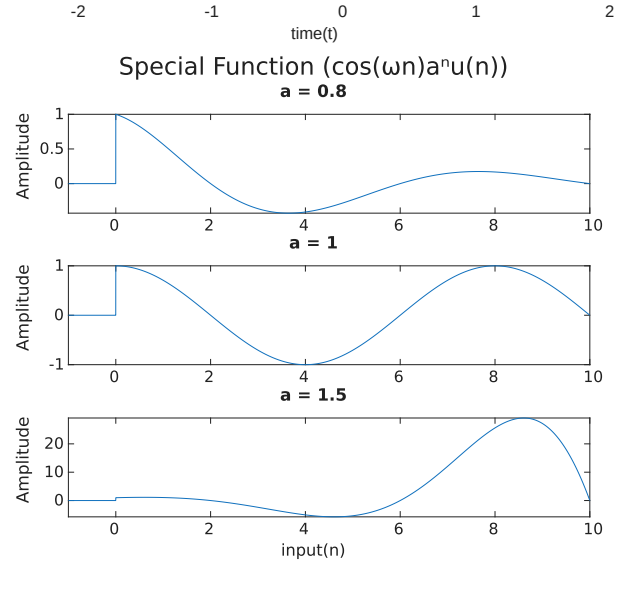
We chose our sampling period to be over the nyquist frequency of the signal, which is around $2 \cdot 2 \cdot 2\pi^2$ since the original function's frequency is around $2\pi^2$, making its energy's frequency around $4\pi^2$, and the nyquist frequency of the enery function about $8\pi^2$. We chose a sampling frequency of 1000, since it is way above our nyquist frequency and within the capabilities of computing machines.

2.6 Special Functions

Submit all three figures, for a total of 8 plots. Also submit the printouts of your Matlab `.m` -files.

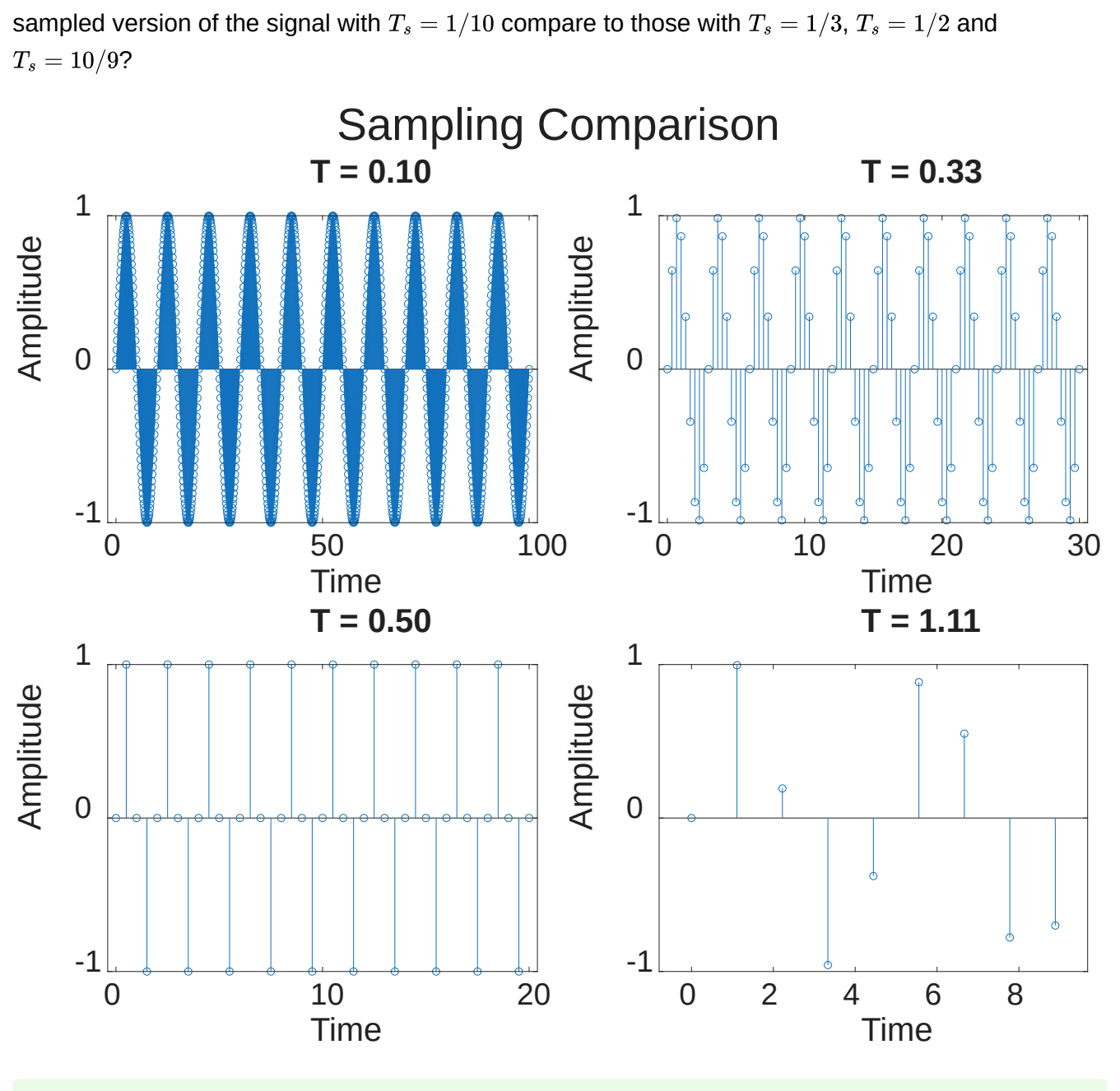






2.7 Sampling

Submit a hardcopy of the figure containing all four subplots. Discuss your results. How does the sampled version of the signal with $T_s = 1/10$ compare to those with $T_s = 1/3$, $T_s = 1/2$ and $T_s = 10/9$?

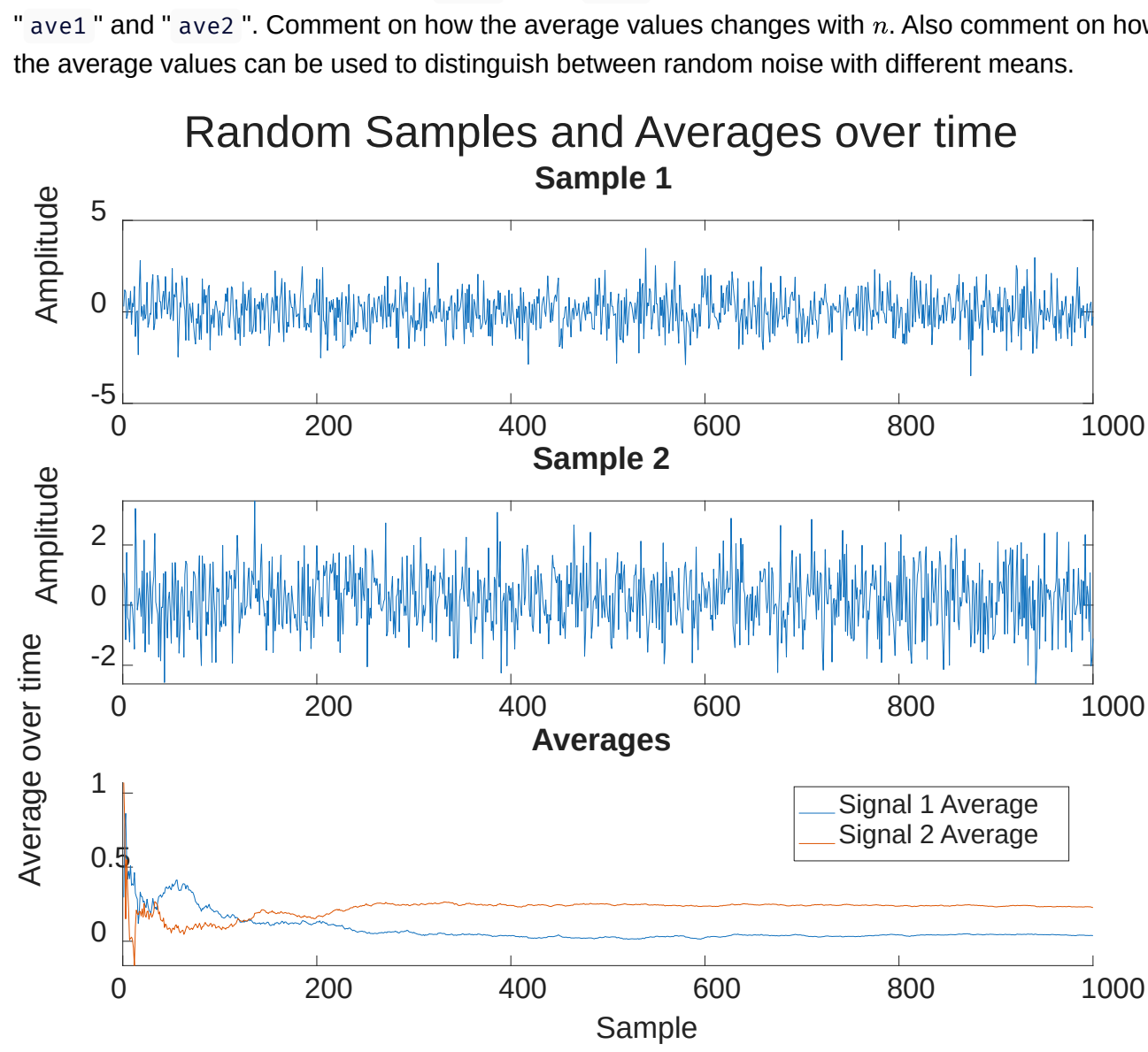


✓ **Answer**

Below the Nyquist frequency, we lose the ability to accurately reconstruct the original $\sin$ frequency. The top two plots are above our Nyquist frequency and are thus easy to reconstruct the original signal. In the bottom left plot we appear to be sampling at exactly the same frequency as the signal itself, being half of the Nyquist, meaning you only see the peaks, troughs, and zeros of the signal itself. For the final plot, we are way below the Nyquist frequency, providing a poor mapping of the signal itself, making it appear to be a lower frequency when sampled.

2.8 Random Signals

Submit your plot of the two signals " `sig1` " and " `sig2` ". Also submit your plot of the two signals " `ave1` " and " `ave2` ". Comment on how the average values changes with n. Also comment on how the average values can be used to distinguish between random noise with different means.

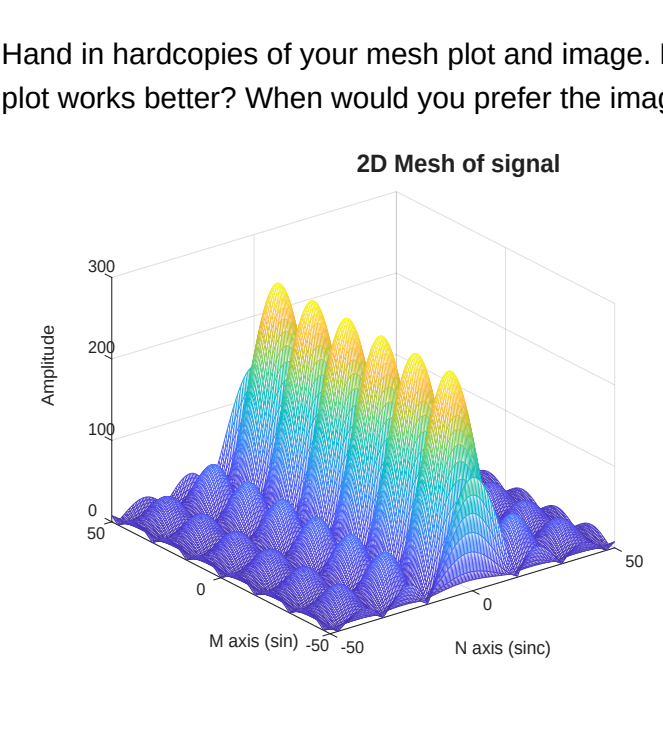


✓ **Answer**

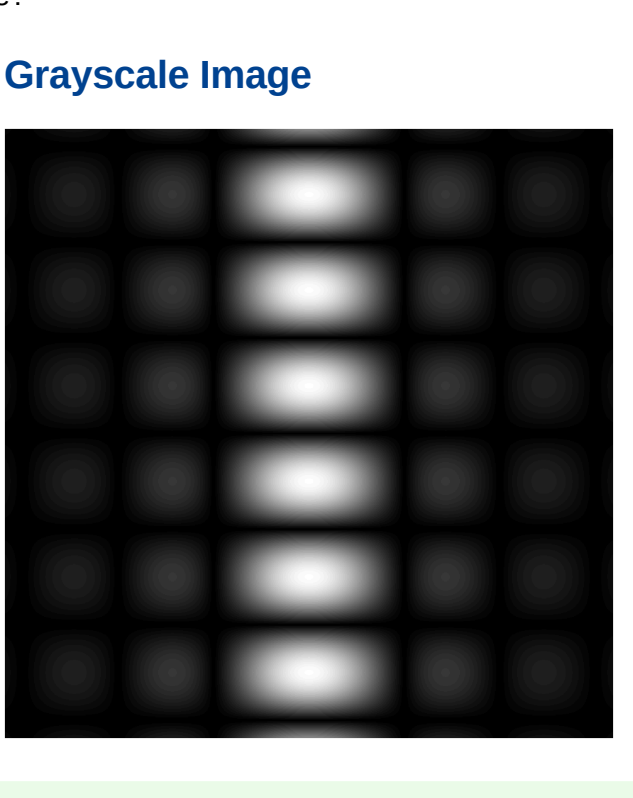
In the beginning of the average plot, both averages are highly uncertain, but as n increases, we notice that the two averages converge to different expected values, with signal 2 at 0.2 and signal 1 at 0. At high n, we can easily distinguish the different means of signals as the averages will converge to their true values.

2.9 2-D Signals

Hand in hardcopies of your mesh plot and image. For which applications do you think the surface plot works better? When would you prefer the image?



Grayscale Image



✓ **Answer**

The mesh plot makes it extremely easy to quickly grok the shape and contour of the 2d signal along with a pleasant visual. As a byproduct however, we are unable to see the full domain of the signal as the 3d shape occludes some of the signal in back of the plot. The image however is easier to postprocess and utilize in other applications.

Code

`helpers.m`

```
% A function to help set our defaults for all plots
function helpers = helpers(f)
    % Disables LaTeX rendering, as it is not supported by the export function
well
    set(f,'defaultTextInterpreter','none')
    set(f,'defaultAxesTickLabelInterpreter','none')
    set(f,'defaultLegendInterpreter','none')
    % Increase Default font size
    defaultFontSize = 25;
    set(f, 'defaultTextFontSize', defaultFontSize);
    set(f, 'defaultAxesFontSize', defaultFontSize);
    set(f, 'defaultLegendFontSize', defaultFontSize);
    % Increase Canvas size
    f.Position(3) = 1000;
    f.Position(4) = 800;
    % Return our font size
    helpers = defaultFontSize;
end
```

`sinc.m`

```
% sinc function defined so that code will work without the signal processing
toolkit
function sinc = sinc(t)
    sinc = sin(t) ./ (t);
    sinc(t == 0) = 1;
end
```

`plot1.m`

```
% Set up canvas
f = figure;
ff = helpers(f);
t = tiledlayout(3,1);

% Plot Discrete plot with stem
nexttile
n = 0:2:60;
y = sin(n/6);
stem(n,y)
ylabel("Amplitude")
title("Discrete sampling")

% Plot high sample continuous
nexttile
n1 = 0:2:60;
z = sin(n1/6);
plot(n1,z)
ylabel("Amplitude")
title("High Sampling")

% Plot low sample continuous
nexttile
n2 = 0:10:60;
w = sin(n2/6);
plot(n2,w)
ylabel("Amplitude")
title("Low Sampling")

% Set labels on toplevel canvas
xlabel(t, "Time", FontSize=ff)
sgtitle("Sin sampling comparison", FontSize=ff*1.5)

% Export graphic
exportgraphics(f, 'plots/1_sin_sampling.svg', ContentType='vector')
```

`plot2.m`

```
% Define input functions and bounds
f1 = @(t) sin(5*t)^2;
f2 = @(t) exp(t);
bounds1 = [0, 2*pi];
bounds2 = [0, 1];

% Define riemann integral helper function
integ_helper = @(N, f, bounds) sum(arrayfun(f, linspace(bounds(1), bounds(2),
N)))/N*(bounds(2)-bounds(1));
% Calculate riemann sum over increasing samples
integ_conv = @(N, f, bounds) arrayfun(@(n) integ_helper(n, f, bounds),
linspace(1, N, N));

% Calculate riemann convergence for both functions
conv1 = integ_conv(100, f1, bounds1);
conv2 = integ_conv(100, f2, bounds2);

% Define functions asked for by questions (unused)
I = @(N) integ_helper(N, f1, bounds1);
J = @(N) integ_helper(N, f2, bounds2);

% Set up canvas
f = figure;
ff = helpers(f);
tiledlayout(2,1)

% Plot first function with riemann convergence
nexttile
plot(conv1)
axis padded
xlabel("Iterations (N)")
ylabel("Sum Value")
title("sin²(5t)")

% Same for second function
nexttile
plot(conv2)
axis padded
xlabel("Iterations (N)")
ylabel("Sum Value")
title("eᵗ")

% Set labels on toplevel canvas
sgtitle("Integral Convergence", FontSize=ff*1.5)

% Export graphic
exportgraphics(f,'plots/2_integral_convergence.svg', ContentType='vector')
```

`plot3.m`

```
% Read audio file
audio = audioread("speech.au");

% Play audio file
sound(audio)

% Set up canvas
f = figure;
ff = helpers(f);

% Plot audio
plot(audio)
xlabel("Time")
ylabel("Amplitude")
title("Audio File")

% Export graphic
exportgraphics(f,'plots/3_audio.svg', ContentType='vector')
```

`plot4.m`

```
% Define signal1 given from assignment
function signal1 = signal1(t)
    signal1 = sin(t./pi) ./ (t ./ pi);
    signal1(t == 0) = 1;
end

% Define constants
bounds = 4*pi^2;
samples = 10000;

% Define domain and range
d = linspace(-bounds, bounds, samples);
vals = signal1(d);

% Riemann sum integrator
integ_helper = @(N, f, bounds) sum(arrayfun(f, linspace(bounds(1), bounds(2),
N)))/N*(bounds(2)-bounds(1));

% Define energy function based on signal1
energy_func = @(t) signal1(t).^2;

% Calculate energy over time and energy total with riemann sun
energy_vals = energy_func(d);
energy_total = integ_helper(samples, energy_func, [-bounds, bounds]);

% Calculate min and max values
[ymax, imax] = max(vals);
[ymin, imin] = min(vals);

% Set up canvas
f = figure;
hold on
ff = helpers(f);

% Plot signal, min, max, energy
p1 = plot(d, vals);
p2 = plot(d(imax), ymax, 'ro', 'MarkerSize',8, 'LineWidth',2);
p3 = plot(d(imin), ymin, 'bo', 'MarkerSize',8, 'LineWidth',2);
p4 = plot(d, energy_vals);

% Set axes to be consistent with period
gca.FontSize = ff;
xticks(-4*pi^2:pi^2:4*pi^2)
xticklabels({'-4π²','-3π²','-2π²','-π²','0','π²','2π²','3π²','4π²'})

% Label all functions
legend( ...
    'signal1(t)', ...
    sprintf('Max = %.4f', ymax), ...
    sprintf('Min = %.4f', ymin), ...
    sprintf('Energy Total = %.4f', energy_total), ...
    Location='best', ...
    FontSize=ff )

xlabel("Time", FontSize=ff)
ylabel("Amplitude", FontSize=ff)
title("Signal 1 and Energy", FontSize=ff*1.5)
hold off

% Export graphic
exportgraphics(f,'plots/4_sinc.svg', ContentType='vector')
```

`plot5.m`

```
% Define constants
samples = 10000;

% Define domains
t1 = linspace(-10*pi,10*pi,samples);
t2 = linspace(-2,2,samples);
t3 = linspace(-20,20,samples);
t4 = linspace(-1,10,samples);

% Define helper functions and ranges for plotting
rect = @(t, T)  abs(t) <= T/2;
f3 = @(a, n) a.^n.*(u(n)-u(n-10));
f4 = @(w, a, n) cos(w.*n).*a.^n.*u(n);
u = @(n) (n>=0);
w = pi/4;
a = [0.8, 1.0, 1.5];
a_title = "a = " + a;

function plot_f(t, f, ti, ylab)
    nexttile
    y = f(t);
    plot(t,y)
    xlim([min(t), max(t)])
    title(ti)
    ylabel(ylab)
end

% Set up canvas
f1 = figure;
ff = helpers(f1);
tt1 = tiledlayout(2,1);
% Plot sinc function
plot_f(t1, @sinc, "sinc(t)", "Amplitude")
% Plot rect function
plot_f(t2, @(t) rect(t, 1), "rect(t)", "Amplitude")
% Label the toplevel
xlabel(tt1, "time(t)", FontSize=ff)
sgtitle("rect and sinc functions", FontSize=ff*1.5)

% Set up canvas
f2 = figure;
ff = helpers(f2);
tt2 = tiledlayout(3,1);
% Plot special function with various a's
arrayfun(@(a, ti) plot_f(t3, @(n) f3(a, n), ti, "Amplitude"), a, a_title)
% Label the toplevel
xlabel(tt2, "input(n)", FontSize=ff)
sgtitle("Special Function (aⁿ(u(n)-u(n-10))", FontSize=ff*1.5)

% Set up canvas
f3 = figure;
ff = helpers(f3);
tt3 = tiledlayout(3,1);
% Plot special function with various as
arrayfun(@(a, ti) plot_f(t4, @(n) f4(w, a, n), ti, "Amplitude"), a, a_title)
% Label the toplevel
xlabel(tt3, "input(n)", FontSize=ff)
sgtitle("Special Function (cos(ωn)aⁿu(n))", FontSize=ff*1.5)

% Export graphics
exportgraphics(f1,'plots/5_special1.svg', ContentType='vector')
exportgraphics(f2,'plots/5_special2.svg', ContentType='vector')
exportgraphics(f3,'plots/5_special3.svg', ContentType='vector')
```

`plot6.m`

```
% Set up canvas
f = figure;
ff = helpers(f);
tiledlayout(2, 2)

% Define sampling frequencies
samp_f = [10, 3, 2, 9/10];

% Define plotting function for each frequency
function plot_f = plot_f(t)
    T = 1/t;
    n = 0:T:(10*t);
    x1 = @(t, n) sin(2.*pi.*t.*n);
    y = x1(T, n);
    stem(n,y)
    xlabel("Time")
    ylabel("Amplitude")
    title(sprintf('T = %.2f', T))
end

% Plot each frequency
for t = 1:numel(samp_f)
    nexttile
    plot_f(samp_f(t))
end

% Label toplevel
sgtitle("Sampling Comparison", FontSize=ff*1.5)

% Export graphic
exportgraphics(f, 'plots/6_sampling_comparison.svg', ContentType='vector')
```

`plot7.m`

```
% Define constants
samples = 1000;

% Sample gaussian randomly for each signal
sig1 = randn(samples, 1);
sig1 = sig1(:);
sig2 = randn(samples, 1) + 0.2;
sig2 = sig2(:);

% Calculate rolling averages
ave1 = cumsum(sig1)' ./ (1:samples);
ave2 = cumsum(sig2)' ./ (1:samples);

% Set up canvas
f = figure;
ff = helpers(f);
t = tiledlayout(3,1);

% Plot sample 1
nexttile
plot(sig1)
ylabel("Amplitude")
title("Sample 1")

% Plot sample 2
nexttile
plot(sig2)
ylabel("Amplitude")
title("Sample 2")

% Plot rolling averages
nexttile
hold on
plot(ave1)
plot(ave2)

% Label averages
legend( ...
    'Signal 1 Average', ...
    'Signal 2 Average', ...
    'Location','best' )

    ylabel("Average over time")
    title("Averages");
hold off

% Label toplevel
xlabel(t, "Sample", FontSize=ff)
sgtitle("Random Samples and Averages over time", FontSize=ff*1.5)

% Export graphic
exportgraphics(f,'plots/7_random.svg', ContentType='vector')
```

`plot8.m`

```
% Define helper function to get a grid
get_grid = @(samples) meshgrid(linspace(-50,50,samples),
linspace(-50,50,samples));

% Define function to plot
f = @(m,n) 255.*abs(sinc(0.2.*m).*sin(0.2.*n));

% Set up canvas and calculate points
fig1 = figure;
[N,M] = get_grid(200);
Z = f(N, M);
ff = helpers(fig1);
% Plot mesh with labels
mesh(N,M,Z);
xlabel('N axis (sinc)');
ylabel('M axis (sin)');
zlabel('Amplitude');
title("2D Mesh of signal", FontSize=ff*1.5)
warning('off', 'MATLAB:print:ContentTypeImageSuggested')
% Export graphic
exportgraphics(fig1,'plots/8_mesh.svg', ContentType='vector')

% Set up canvas and calculalte points
fig2 = figure;
n = 1000;
[N,M] = get_grid(n);
Z = f(N, M);
ff = helpers(fig2);
% Create grayscale image
imagesc(n,n,Z)
axis image off
colormap(gray(256));
% Export graphic
exportgraphics(fig2,'plots/8_image.png');
```